

Underwater Acoustic Model for Whale Species

Course: EGN 4952C-001: Engineering Design 2, Florida Atlantic University

Prepared by: Ace Bowen, Fahima Aziz, Eoin Nash, Khang Vinh Doan

Sponsor: Dr. Ali Ibrahim | Faculty Advisor: Dr. Waseem Asghar | Date: Spring 2026 Semester

Final Project Video Presentation: <https://youtu.be/GM90G2w2DNk>

1. Project Overview

This project is a local AI-based whale audio identification system. It is designed to help identify whale species from underwater acoustic recordings and support marine conservation, research, and monitoring. The system converts uploaded audio into spectrogram-style features, uses trained CNN models, and returns a species label with confidence information.

The final package includes a Python backend, React/Vite frontend, trained PyTorch model weights, label encoders, evaluation files, labeled datasets, training notebook, team documentation, and poster materials. It should be treated as a working prototype rather than a production deployment.

2. System Workflow

Step	Description
1. Upload	User uploads audio through the local web interface.
2. Preprocess	Backend decodes audio, converts to mono, resamples, builds mel-spectrogram, normalizes, and resizes input.
3. Gate Model	First CNN checks whether the sound is whale-like enough to continue.
4. Species Model	Second CNN predicts the most likely class/species label.
5. Output	Frontend displays species prediction, confidence, and audio analysis values.

3. Technology Stack

Area	Tools / Files
Backend	Python, FastAPI, PyTorch, audio preprocessing libraries
Frontend	React, Vite, JavaScript, CSS
Machine Learning	CNN models, mel-spectrogram input, .pth model weights
Metrics / Encoders	.pkl label encoders, .csv training history files, .npy confusion matrices
Dataset	Labeled WAV audio clips organized by species/class folders
Documentation	README files, group minutes, poster, notebook

4. Main Files

File / Folder	Purpose
app.py	FastAPI backend for loading models, processing audio, and serving predictions.
DiffusionWhaleClassifier.ipynb	Training/development notebook for model experimentation.
whale_gate_classifier.pth	Saved gate model used before species classification.
whale_species_classifier.pth	Saved species classification model.
*.pkl label encoders	Label mapping files for model outputs.
*_training_history.csv	Training history files.
_confusion_matrix.npy	Confusion matrix evaluation files.
datasets/	Labeled audio clips and class metadata.
whales_website/	React/Vite frontend application.
GROUPMINUTES-Grp10.xlsx	Team meeting/minutes documentation.
SPRING_26_SNR_SHOWCASE_POSTER_GRP10.pptx	Project showcase poster.
videoLink.txt	This file contains the video link.

5. Dataset Summary

The submitted archive contains 697 WAV clips across the following class folders:

Class Folder	WAV Clips
Beluga_White_Whale	46
Bowhead_Whale	45
Clymene_Dolphin	41
False_Killer_Whale	42
Finback_Whale	36
Humpback_Whale	50

Killer_Whale	27
Long-Finned_Pilot_Whale	59
Narwhal	40
Northern_Right_Whale	150
Rough-Toothed_Dolphin	39
Short-Finned_Pacific_Pilot_Whale	50
Southern_Right_Whale	20
Sperm_Whale	52

Some dolphin or dolphin-like classes are included because the project uses a gate/classification approach that helps separate whale-like audio from non-target or acoustically similar marine mammal sounds.

6. Directory Map

```

ED2-26s_g10/
|-- app.py
|-- DiffusionWhaleClassifier.ipynb
|-- diffusion_unet.pth
|-- full_label_encoder.pkl
|-- gate_confusion_matrix.npy
|-- gate_confusion_matrix_normalized.npy
|-- gate_training_history.csv
|-- GROUPMINUTES-Grp10.xlsx
|-- RUN_ME.md
|-- SHARE_CHECKLIST.md
|-- species_confusion_matrix.npy
|-- species_confusion_matrix_normalized.npy
|-- SPRING_26_SNR_SHOWCASE_POSTER_GRP10.pptx
|-- training_history_diffusion.csv
|-- videoLinks.txt
|-- whale_gate_classifier.pth
|-- whale_species_classifier.pth
|-- whale_species_label_encoder.pkl
|-- datasets/
|   |-- Beluga_White_Whale/
|   |-- Bowhead_Whale/
|   |-- Clymene_Dolphin/
|   |-- False_Killer_Whale/
|   |-- Finback_Whale/
|   |-- Humpback_Whale/
|   |-- Killer_Whale/
|   |-- Long-Finned_Pilot_Whale/
|   |-- Narwhal/
|   |-- Northern_Right_Whale/
|   |-- Rough-Toothed_Dolphin/
|   |-- Short-Finned_Pacific_Pilot_Whale/
|   |-- Southern_Right_Whale/
|   |-- Sperm_Whale/
|-- whales_website/
|   |-- index.html
|   |-- package.json
|   |-- package-lock.json
|   |-- img/
|   |-- public/
|   |-- src/

```

7. Setup Requirements

Install Python 3.10 or newer, Node.js 18 or newer, npm, and optionally FFmpeg for compressed/non-WAV audio support. A CPU is enough for basic inference, but a CUDA-capable GPU is recommended for training or heavier testing.

```
pip install fastapi uvicorn torch torchaudio librosa numpy scipy
```

```
pip install pandas scikit-learn joblib python-multipart soundfile pydub
```

8. How to Run the Project

Backend

```
cd ED2-26s_g10
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
# macOS/Linux:
source .venv/bin/activate
pip install fastapi uvicorn torch torchaudio librosa numpy scipy
pip install pandas scikit-learn joblib python-multipart soundfile pydub python app.py
```

Frontend

```
cd ED2-26s_g10/whales_website
npm install
npm run dev
# Open the Vite URL, usually http://localhost:5173
```

9. API Routes

Route	Purpose
GET /	Backend health/status response.
GET /classes	Returns available species/class labels.
POST /predict	Accepts uploaded audio and returns prediction JSON.

10. Testing and Verification

Start the backend, confirm GET / and GET /classes work, start the frontend, upload a WAV file from one of the dataset folders, and compare the returned prediction with the folder label. Review the training history CSV files and confusion matrix NPY files for model performance evidence.

Reported prototype validation results from project materials include approximately 95.04% gate model validation accuracy and

93.39% species model validation accuracy. These should be treated as prototype validation results, not proof of productionlevel reliability.

11. Known Limitations

Real ocean environments include vessel noise, environmental interference, overlapping calls, and recording differences that may reduce accuracy. Some folders have fewer examples than others, so class imbalance can affect results. The app is designed for local prototype demonstration and is not hardened for public deployment or scientific decision-making without expert review.

12. Equipment and Software Audit

No physical equipment is used except for personal computers; our project is software-based.

References / Data Sources

NOAA DCLDE/HICEAS towed array data; Kaggle Marinexplore and Cornell University Whale Detection Challenge; Zenodo whale sound dataset search results; PyTorch, FastAPI, React, Vite, and Librosa documentation.